

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN MƯỜI BỐN
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã ổn định các luồng vận hành chính của hệ thống trong tuần thứ mười ba, bao gồm RTSP camera, refresh token, phân quyền người dùng, xử lý lỗi xác thực và chuẩn hóa datetime trong môi trường SQLite, tuần thứ mười bốn của dự án tập trung vào một thay đổi quan trọng ở tầng dữ liệu: chuyển hệ quản trị cơ sở dữ liệu quan hệ từ SQLite sang PostgreSQL.

Ở các giai đoạn trước, SQLite phù hợp cho quá trình phát triển nhanh, thử nghiệm chức năng và chạy local. Tuy nhiên, khi hệ thống đã có nhiều module quan trọng như Auth, User Management, Camera Management, Audit Log, Alert, Refresh Token và các workflow nghiệp vụ phức tạp hơn, SQLite bắt đầu bộc lộ hạn chế về khả năng mở rộng, quản lý schema và tính phù hợp với môi trường production.

Vì vậy, trong tuần này em tập trung thực hiện quá trình migration relational database sang PostgreSQL, đồng thời chuẩn hóa lại cách quản lý schema bằng Alembic thay vì để ứng dụng tự tạo bảng khi khởi động.

Các mục tiêu chính trong tuần này bao gồm:

- Chuyển cấu hình relational database từ SQLite sang PostgreSQL
- Tách quá trình tạo schema khỏi runtime startup của ứng dụng
- Tích hợp Alembic để quản lý version migration cho database
- Chuẩn hóa toàn bộ timestamp trong SQL database theo timezone-aware datetime
- Bổ sung dữ liệu cũ từ SQLite sang PostgreSQL
- Cập nhật dependency và tài liệu cấu hình cần thiết cho PostgreSQL

Việc hoàn thành các mục tiêu này giúp hệ thống tiến thêm một bước quan trọng từ môi trường phát triển local sang kiến trúc backend có khả năng bảo trì, mở rộng và triển khai thực tế tốt hơn.

2. Chuyển relational database từ SQLite sang PostgreSQL

2.1. Lý do cần chuyển đổi database

Trong các tuần trước, hệ thống sử dụng SQLite để lưu trữ các dữ liệu quan hệ như người dùng, refresh token, camera, alert và audit log. Cách tiếp cận này giúp việc phát triển ban đầu đơn giản hơn vì SQLite không yêu cầu cài đặt server riêng và dễ chạy trên máy cá nhân.

Tuy nhiên, khi hệ thống ngày càng có nhiều nghiệp vụ hơn, SQLite không còn thật sự phù hợp cho định hướng production vì một số lý do:

- SQLite chủ yếu phù hợp cho môi trường local hoặc ứng dụng nhỏ
- Khả năng xử lý nhiều kết nối đồng thời hạn chế hơn PostgreSQL
- Không phù hợp bằng PostgreSQL trong các hệ thống backend có nhiều người dùng
- Quản lý migration schema về lâu dài khó kiểm soát nếu chỉ dùng `create_all()`
- Không phản ánh sát môi trường triển khai thực tế

Trong khi đó, PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ phổ biến, mạnh mẽ và phù hợp hơn cho hệ thống backend thực tế. Việc chuyển sang PostgreSQL giúp hệ thống có nền tảng dữ liệu ổn định hơn, dễ mở rộng hơn và phù hợp với các workflow như authentication, audit log, camera management và refresh token.

2.2. Cập nhật cấu hình database

Trước đây, hệ thống sử dụng biến cấu hình: `DATABASE_URL`

Sau tuần này, cấu hình được chuyển sang: `POSTGRES_URL`

Việc đổi tên biến cấu hình giúp thể hiện rõ hệ thống hiện sử dụng PostgreSQL làm relational database chính, tránh nhầm lẫn với SQLite cũ.

Một điểm quan trọng là `connect_args={"check_same_thread": False}` đã được loại bỏ. Lý do là tham số này chỉ dành riêng cho SQLite nhằm cho phép kết nối được dùng giữa nhiều thread khác nhau. PostgreSQL sử dụng driver riêng như `psycopg2`, do đó không sử dụng cấu hình này.

Thay đổi này giúp cấu hình database trở nên đúng hơn với PostgreSQL và tránh các lỗi không tương thích khi khởi tạo engine.

2.3. Cập nhật file môi trường và dependency

Để hỗ trợ PostgreSQL, file `.env.example` được cập nhật thêm cấu hình mới cho `POSTGRES_URL`.

Bên cạnh đó, file `requirements.txt` cũng được bổ sung các dependency cần thiết:

- `alembic`
- `psycopg2-binary`

Trong đó:

- psycopg2-binary là driver giúp SQLAlchemy/SQLModel kết nối tới PostgreSQL.
- alembic là công cụ quản lý migration schema cho SQLAlchemy.

Việc bổ sung các dependency này là bước cần thiết để hệ thống có thể hoạt động với PostgreSQL và quản lý schema theo version một cách rõ ràng.

3. Tách schema migration khỏi runtime startup

3.1. Vấn đề với cách dùng `create_all()`

Trước đây, khi ứng dụng khởi động, hệ thống gọi:

`SQLModel.metadata.create_all(engine)`

Cách này giúp tự động tạo bảng nếu bảng chưa tồn tại. Tuy nhiên, khi hệ thống còn nhỏ thì cách làm này khá tiện, nhưng về lâu dài lại có nhiều hạn chế.

Một số vấn đề có thể xảy ra:

- Schema có thể bị tạo tự phát ngoài kiểm soát
- Khó theo dõi lịch sử thay đổi của database
- Khó đồng bộ schema giữa các máy khác nhau
- Không phù hợp với quy trình production
- Không quản lý rõ được các thay đổi như thêm cột, đổi kiểu dữ liệu hoặc sửa constraint

Vì vậy, trong tuần này em đã tách việc tạo schema ra khỏi runtime startup. Ứng dụng không còn tự tạo bảng bằng `create_all()` khi chạy nữa.

3.2. Runtime startup chỉ còn seed dữ liệu ban đầu

Sau khi loại bỏ `create_all()`, hàm khởi tạo database được điều chỉnh lại.

Trước đây: `init_sql_db()`

Hàm này vừa tạo bảng, vừa tạo initial admin.

Sau khi refactor: `init_initial_data()`

Runtime startup hiện chỉ còn nhiệm vụ kiểm tra và tạo dữ liệu ban đầu cần thiết, ví dụ như tài khoản admin mặc định nếu chưa tồn tại.

Việc quản lý schema được chuyển hoàn toàn sang Alembic. Điều này giúp trách nhiệm của từng phần rõ ràng hơn:

- Alembic chịu trách nhiệm tạo và cập nhật schema
- Application startup chỉ chịu trách nhiệm khởi tạo dữ liệu ban đầu

Cách tách biệt này phù hợp hơn với một hệ thống backend thực tế.

4. Tích hợp Alembic để quản lý schema

4.1. Khởi tạo Alembic trong Backend

Trong tuần này, em đã tích hợp Alembic vào thư mục Backend.

Các file và thư mục mới bao gồm:

- SourceCode/BE/alembic.ini
- SourceCode/BE/alembic/env.py
- SourceCode/BE/alembic/script.py.mako
- SourceCode/BE/alembic/versions/

Alembic đóng vai trò quản lý toàn bộ lịch sử thay đổi schema của relational database. Thay vì để ứng dụng tự tạo bảng, mỗi thay đổi về schema sẽ được biểu diễn bằng một migration file cụ thể.

Điều này giúp hệ thống có thể kiểm soát được:

- Schema hiện tại đang ở version nào
- Đã từng có những thay đổi gì
- Cần upgrade hoặc downgrade database ra sao

Cách tiếp cận này đặc biệt quan trọng khi dự án có nhiều bảng như user, refresh token, camera, audit log, alert và có thể tiếp tục mở rộng trong tương lai.

4.2. Tạo migration initial schema

Migration đầu tiên được tạo là: **675746483be3_initial_schema.py**

Migration này đại diện cho schema khởi tạo của hệ thống relational database trên PostgreSQL.

Nó có nhiệm vụ tạo các bảng chính của hệ thống, bao gồm các nhóm dữ liệu như:

- User
- Refresh token
- Camera
- Audit log
- Alert

- Các bảng liên quan khác trong SQLAlchemy metadata

Việc có migration initial schema giúp quá trình dựng database trên máy mới trở nên rõ ràng hơn. Sau khi clone project hoặc reset database, chỉ cần chạy:

1. `cd SourceCode/BE`
2. `alembic upgrade head`

Database sẽ được tạo theo đúng schema đã version hóa

4.3. Migration timezone-aware datetime

Migration tiếp theo là: **315f624a3d58_make_timestamps_timezone_aware.py**

Migration này tập trung vào việc chuẩn hóa các field datetime sang kiểu có timezone.

Lý do cần migration này là PostgreSQL hỗ trợ kiểu dữ liệu `TIMESTAMP WITH TIME ZONE`. Đây là kiểu phù hợp hơn cho các dữ liệu biểu diễn thời điểm thật trong hệ thống, đặc biệt với các trường như thời điểm tạo tài khoản, thời điểm token hết hạn, thời điểm camera bị xóa, thời điểm alert được tạo hoặc thời điểm refresh token bị revoke.

Các nhóm field được xử lý gồm:

- `created_at`
- `updated_at`
- `deleted_at`
- `expires_at`
- `revoked_at`
- `last_used_at`
- `last_checked_at`
- `timestamp`
- `reset_token_expires`

Việc chuẩn hóa timezone giúp giảm nguy cơ lỗi khi so sánh thời gian giữa backend, database và token.

5. Chuẩn hóa timestamp và authentication logic

5.1. Chuyển từ UTC naive sang UTC aware

Trong tuần trước, hệ thống đã bắt đầu xử lý các lỗi liên quan đến datetime trong SQLite. Tuy nhiên, khi chuyển sang PostgreSQL, việc chuẩn hóa datetime càng trở nên quan trọng hơn.

- Trước đây, helper `utc_now()` trả về UTC naive datetime
- Sau tuần này, helper được cập nhật thành timezone-aware datetime

Điều này có nghĩa là mọi thời điểm sinh ra từ hệ thống đều giữ lại thông tin timezone UTC.

Đồng thời, các helper cũ cũng được thay thế bằng cách dùng thống nhất hơn.

Việc thống nhất helper thời gian giúp code rõ ràng hơn và tránh nhầm lẫn giữa naive datetime và aware datetime.

5.2. Cập nhật các model datetime

Các model SQLAlchemy như `user`, `refresh token`, `camera`, `audit log` và `alert` được cập nhật để các cột datetime sử dụng:

`DateTime(timezone=True)`

Điều này giúp SQLAlchemy tạo cột PostgreSQL phù hợp với timezone-aware datetime.

Các field như `created_at`, `updated_at`, `deleted_at`, `expires_at`, `revoked_at`, `last_used_at`, `last_checked_at`, `timestamp` và `reset_token_expires` đều được xử lý lại để đồng bộ với PostgreSQL.

Việc thay đổi này giúp dữ liệu thời gian trong database có ý nghĩa rõ ràng hơn và tránh các lỗi phát sinh khi hệ thống cần so sánh thời gian hiện tại với thời gian hết hạn token hoặc thời gian reset password.

6. Kết quả đạt được trong tuần 14

Sau khi hoàn thành các công việc trong tuần thứ mười bốn, các kết quả chính đạt được bao gồm:

- Chuyển relational database từ SQLite sang PostgreSQL
 - Cập nhật cấu hình database sang `POSTGRES_URL`
 - Loại bỏ cấu hình SQLite-only khỏi SQLAlchemy engine

- Tách việc tạo schema khỏi runtime startup của ứng dụng
- Tích hợp Alembic để quản lý version migration cho database
 - Tạo migration initial schema cho hệ thống SQL
 - Tạo migration chuẩn hóa timestamp theo timezone-aware datetime
 - Cập nhật các model SQLAlchemy để sử dụng timezone
 - Chuẩn hóa helper thời gian từ UTC naive sang UTC aware
- Cải thiện config CORS_ORIGINS để hỗ trợ JSON array
- Cải thiện cách resolve MODEL_PATH để không làm fail command Alembic
- Bổ sung dependency alembic và psycopg2-binary
- Chuyển dữ liệu từ SQLite sang PostgreSQL

Những kết quả này giúp hệ thống có nền tảng relational database ổn định hơn, có khả năng quản lý schema rõ ràng hơn và phù hợp hơn với hướng triển khai production.

7. Khó khăn gặp phải và cách khắc phục

Trong tuần thứ mười bốn, quá trình chuyển hệ thống từ SQLite sang PostgreSQL và tích hợp Alembic để quản lý schema đã phát sinh một số vấn đề thực tế trong quá trình cấu hình và chạy lệnh migration. Các khó khăn chủ yếu không nằm ở nghiệp vụ chính của hệ thống, mà xuất hiện ở tầng cấu hình runtime, đặc biệt khi một số phần cấu hình của ứng dụng bị phụ thuộc vào các thành phần không liên quan trực tiếp đến database migration.

7.1. Alembic command bị phụ thuộc vào model AI

Một khó khăn thực tế phát sinh trong quá trình tích hợp Alembic là việc chạy các command migration bị ảnh hưởng bởi cấu hình model AI.

Alembic cần import các module cấu hình và model của ứng dụng để đọc metadata từ SQLAlchemy. Tuy nhiên, trong quá trình import config, hệ thống cũng kiểm tra đường dẫn MODEL_PATH. Điều này dẫn đến tình trạng: nếu file model AI chưa tồn tại, đặt sai vị trí hoặc môi trường chỉ đang chạy migration database mà chưa chuẩn bị model, lệnh Alembic vẫn có thể bị lỗi.

Vấn đề này không hợp lý vì migration database và model AI là hai thành phần độc lập. Việc tạo bảng hoặc cập nhật schema không nên bị phụ thuộc vào việc đã có file trọng số YOLO hay chưa.

Để khắc phục, em đã điều chỉnh lại cách xử lý MODEL_PATH trong cấu hình hệ thống. Thay vì bắt buộc model phải tồn tại ngay tại thời điểm import config, hệ thống được cập nhật để resolve đường dẫn model linh hoạt hơn theo nhiều vị trí khác nhau như thư mục chạy hiện tại, backend root hoặc repo root. Đồng thời, việc không tìm thấy model sẽ không làm hỏng ngay các command Alembic.

Cách xử lý này giúp:

- Các lệnh Alembic có thể chạy độc lập với AI model
- Quá trình setup database trên môi trường mới dễ dàng hơn
- Giảm sự phụ thuộc không cần thiết giữa database migration và inference runtime
- Phù hợp hơn với quy trình triển khai thực tế

7.2. Lỗi parse CORS_ORIGINS khi cấu hình nhiều origin

Một khó khăn khác phát sinh trong quá trình chuẩn hóa cấu hình môi trường là cách parse biến CORS_ORIGINS.

Ban đầu, hệ thống xử lý CORS_ORIGINS bằng cách tách chuỗi theo dấu phẩy. Cách làm này hoạt động tốt với cấu hình đơn giản như:

CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

Tuy nhiên, khi muốn cấu hình theo dạng JSON array trong file .env, ví dụ:

CORS_ORIGINS=["http://localhost:5173","http://127.0.0.1:5173"]

Việc split trực tiếp theo dấu phẩy sẽ làm sai định dạng dữ liệu. Hệ thống có thể hiểu sai từng phần tử vì chuỗi JSON cũng chứa dấu phẩy để phân tách các phần tử trong mảng.

Để khắc phục, em đã cập nhật lại parser cho CORS_ORIGINS. Cơ chế mới kiểm tra nếu giá trị cấu hình bắt đầu bằng dấu [thì sẽ parse theo JSON array. Nếu không, hệ thống mới fallback về cách parse cũ bằng dấu phẩy.

Cách xử lý này giúp hệ thống hỗ trợ được cả hai kiểu cấu hình:

- CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
- CORS_ORIGINS=["http://localhost:5173","http://127.0.0.1:5173"]

Kết quả là phần cấu hình CORS trở nên linh hoạt hơn, ít lỗi hơn khi triển khai ở nhiều môi trường khác nhau như local, staging hoặc production.

8. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã hoàn thành migration relational database sang PostgreSQL và đưa Alembic vào quản lý schema trong tuần thứ mười bốn, hệ thống đã tiến gần hơn tới kiến trúc backend có thể bảo trì và triển khai thực tế. Trong tuần tiếp theo, em sẽ tập trung vào việc kiểm thử toàn bộ hệ thống trên PostgreSQL, hoàn thiện cấu hình triển khai và chuẩn bị cho giai đoạn cuối cùng..

8.1. Kiểm thử toàn bộ API với PostgreSQL

Trong tuần tiếp theo, em sẽ kiểm thử lại toàn bộ các module đang sử dụng relational database để đảm bảo việc chuyển từ SQLite sang PostgreSQL không làm ảnh hưởng đến chức năng.

Các luồng cần kiểm thử bao gồm:

- Đăng ký, đăng nhập và refresh token
- Reset password và verify email
- User Management
- Camera Management
- Audit Log
- Alert
- Phân quyền Admin/Guard
- Các workflow liên quan đến token expiry và revoked token

Mục tiêu là đảm bảo mọi chức năng từng chạy ổn định trên SQLite vẫn hoạt động đúng trên PostgreSQL.

8.2. Chuẩn hóa môi trường triển khai

Sau khi đã chuyển sang PostgreSQL, hệ thống cần được chuẩn hóa môi trường chạy để tránh lỗi cấu hình giữa các máy.

Các công việc dự kiến bao gồm:

- Bổ sung hướng dẫn chạy Alembic
- Cập nhật README với luồng setup mới

- Kiểm tra lại .env.example

Mục tiêu là giúp người khác có thể clone project, cài đặt và chạy hệ thống theo đúng thứ tự mà không gặp lỗi thiếu schema hoặc sai cấu hình database

8.3. Chuẩn bị báo cáo cuối kỳ

Cuối cùng, em sẽ tiếp tục chuẩn bị cho giai đoạn hoàn thiện sản phẩm và báo cáo cuối kỳ.

Các công việc bao gồm:

- Kiểm thử end-to-end toàn bộ hệ thống
- Hoàn thiện tài liệu báo cáo cuối kì
- Cập nhật slide thuyết trình
- Chuẩn bị video demo
- Kiểm tra lại dữ liệu demo
- Rà soát các lỗi nhỏ ở Frontend và Backend

Đây là bước quan trọng để đảm bảo hệ thống không chỉ hoàn thiện về mặt chức năng, mà còn có thể trình bày rõ ràng và ổn định trong buổi báo cáo cuối kỳ.